Hidden Markov Models for Human Activity Recognition

BACKGROUND AND MOTIVATION

I chose a simple wellness use case: knowing whether I am moving (walking), upright but not moving much (standing), doing high-intensity movement (jumping), or completely inactive (phone lying still on a desk). My phone already has motion sensors, so activity can be estimated without GPS or the camera. The challenge is that sensor readings are noisy and do not label themselves. I only see acceleration and rotation, not the activity name. A Hidden Markov Model (HMM) fits this problem well: the true activity is hidden, each time step produces a measurement I compute from the sensors, and the model learns how activities tend to continue or switch over time. This report describes how I collected data, built features, trained the model, and tested it on new recordings.

DATA COLLECTION AND PRE-PROCESSING STEPS

How I collected data

I used the Sensor Logger app on my Google Pixel 3a (Android). For each trial I recorded roughly 5–10 seconds of:

Activity	**What I did**
Standing	Held the phone steady at waist level
Walking	Walked indoors at a steady pace
Jumping	Performed continuous jumps
Still	Placed the phone flat on a table with no movement

Each export includes accelerometer and gyroscope values on three axes (x, y, z), plus timestamps. I saved and organized files by activity name in the project dataset folder (data/processed/).

Video evidence

Each activity has a separate about 5 second video showing my full body, the phone, and my face (well lit). Paste your links in the Video link column:

Activity	Video link
Standing	Standing
Walking	Walking
Jumping	Jumping
Still	Still

Dataset size and quality

After cleaning and labeling, I kept 85 usable CSV files:

Activity	**Number of files**	**Total recording time (approx.)**
Standing	24	about 3 min 27 s
Walking	21	about 2 min 40 s
Jumping	20	about 2 min 29 s
Still	20	about 2 min 53 s

I prepared 85 labeled CSV files in data/processed/ (Standing 24, Walking 21, Jumping 20, Still 20). Total time per activity is above 1 min 30 s. I merged the accelerometer and gyroscope by time and resampled all clips to 100 Hz (native rate on my phone: 100 Hz, 10 ms per sample) so each 1 s window has the same number of samples.

Windowing: 100 samples (1.0 s) window, 50 samples (0.5 s) hop. One second fits roughly one stride or jump cycle and gives several windows per clip.

Features (12 per window): time-domain summaries of acceleration and gyroscope (mean, variance, std, RMS, SMA, axis correlation, gyro stats) plus three FFT-based values on acceleration magnitude (dominant frequency, spectral energy, peak ratio). I z-score normalized features using training data only.

HMM SETUP AND IMPLEMENTATION

Hidden states: Standing, Walking, Jumping, Still.

Observations: 12 normalized features per window.

A: transition probabilities between activities; B: Gaussian emissions per state; pi: initial state probabilities.

Implemented in NumPy (src/hmm.py), run from notebook/hmm_activity_recognition_full.ipynb.

Baum–Welch trains A, B, and pi; training stops when log-likelihood gain is below 0.0001.

Viterbi decodes the most likely activity path per clip; outputs are used for sensitivity, specificity, and the confusion matrix.

RESULTS AND INTERPRETATION

Unseen test clips (not used in training or normalization):

Standing10-2026-10-28_18-54-27

Walking19-2026-10-29_17-27-45

Jumping19-2026-10-29_17-21-17

Still20-2026-10-28_19-39-51

81 files for training, 4 for testing (same phone, later sessions).

Performance (window-level, Viterbi):

State (Activity)	Samples	Sensitivity	Specificity	Overall Accuracy
Standing	14	1.0000	1.0000	1.0000
Walking	12	1.0000	1.0000	1.0000
Jumping	15	1.0000	1.0000	1.0000
Still	13	1.0000	1.0000	1.0000

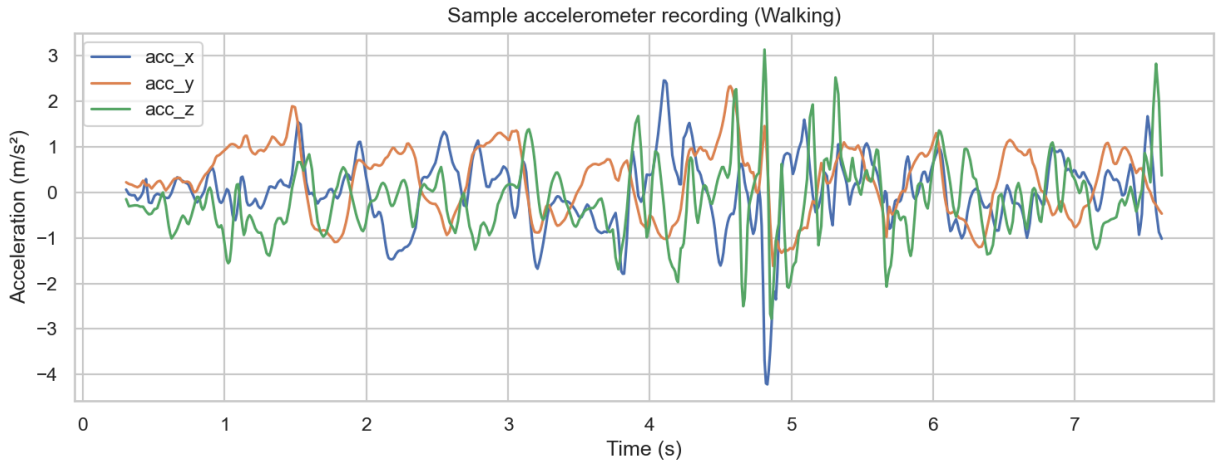
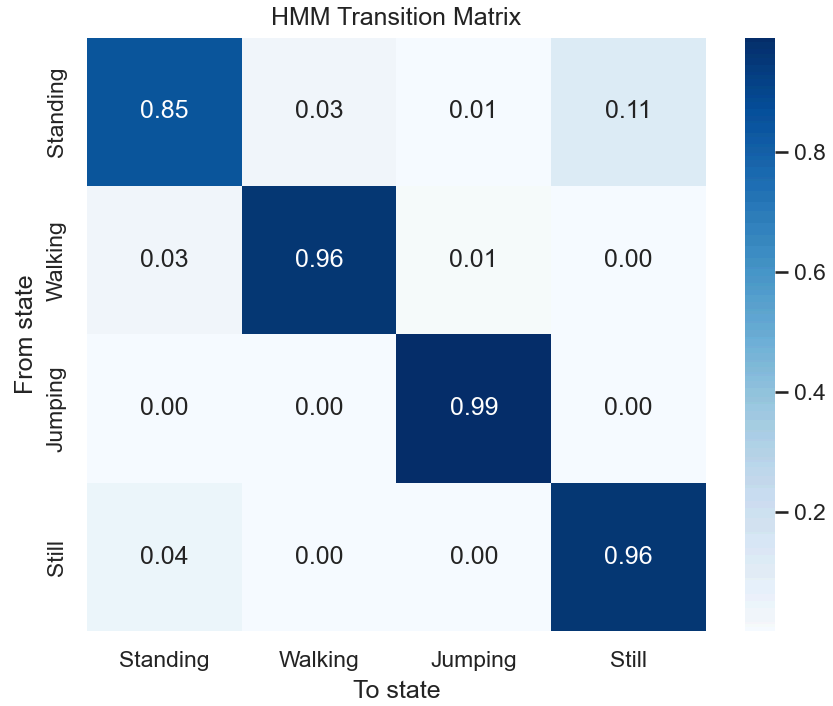


Figure 1- raw_accelerometer_sample.png



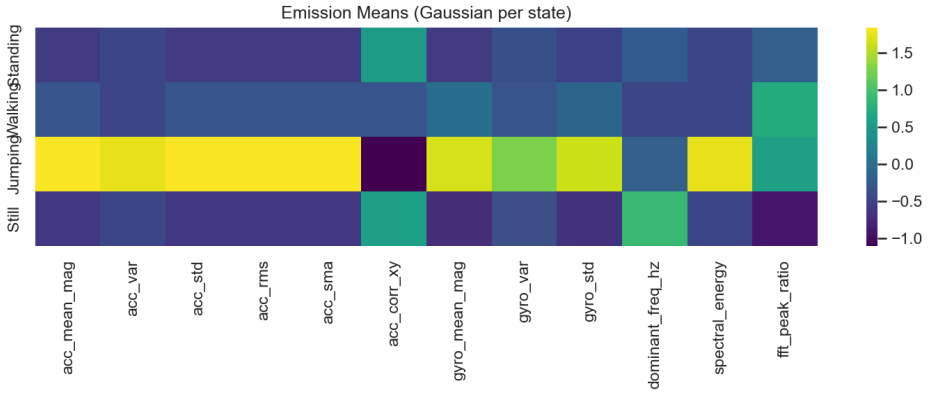


Figure 2 - transition_matrix.png

Figure 3 -emission_means.png

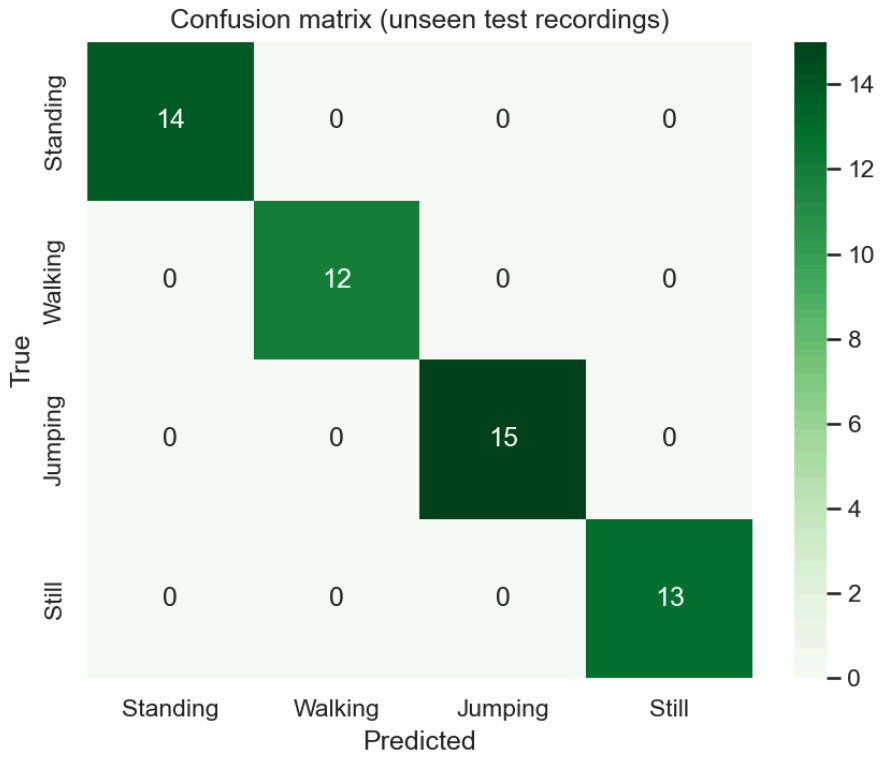
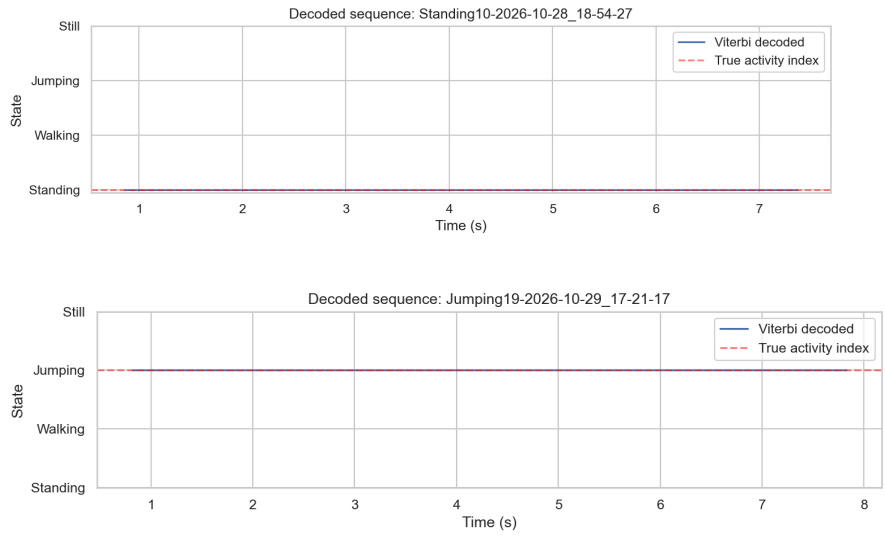
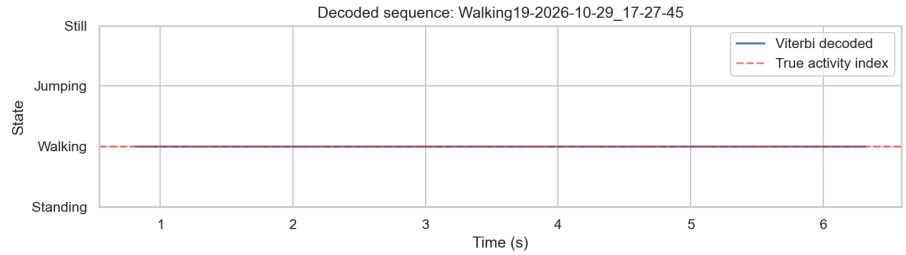
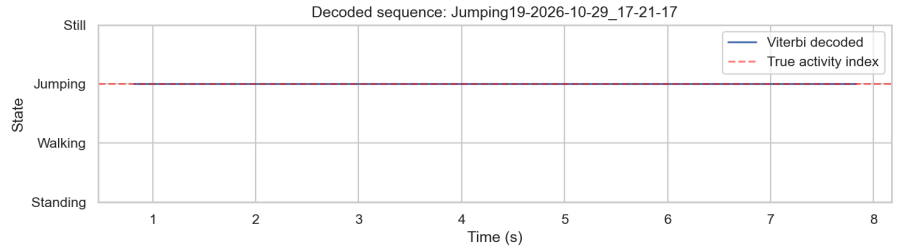


Figure 3 -Confusion_matrix.png







Transition matrix: strong diagonal (activity stays the same between windows). Emissions: walking/jumping show higher motion energy; standing vs still differ in gyro and variance. Confusion matrix on test data: all windows classified correctly in this run.

DISCUSSION AND CONCLUSION

Walking and jumping were easiest (clear periodic motion). Standing vs still was hardest (both low motion), gyro and orientation features helped. Transition probabilities match short single-activity clips (high self-transitions). Harmonizing to 100 Hz and 1 s windows reduced noise effects. Future work: more data, longer multi-activity sessions, and testing on another person or device.

I completed a pipeline from CSV exports to a trained HMM with Baum–Welch and Viterbi; held-out clips were classified correctly at window level in my evaluation.